

MeshCore Port

Arduino MKR WAN 1310 LoRa

Contents

1. Prerequisite	3
1.1. What you need on the bench	3
1.2. Two brains in one board	3
1.3. What we originally planned to do	3
2. The key discovery: dumb mode	4
2.1. What this changes about the project	4
3. What the board gives us, and what it hides	6
3.1. The pin map	6
3.2. Two things that follow from the wiring	6
3.3. Three wires we do not get	7
4. Other routes we considered	8
5. Phase 0: prove dumb mode actually works	9
5.1. Step 0.1: establish the way back first	9
5.2. Step 0.2: knock on the door	9
5.3. Step 0.3: transmit	10
5.4. Step 0.4: receive	10
5.5. Step 0.5: measure how good the radio path really is	10
5.6. Step 0.6: the backup plan, a tiny helper firmware	11
5.7. When Phase 0 is done	12
6. Phases 1 to 5	13
6.1. Phase 1: teach MeshCore's build system about SAMD21	13
6.2. Phase 2: board support and storage	13
6.3. Phase 3: the board definition	13
6.4. Phase 4: fix the channel check	14
6.5. Phase 5: real world testing and write up	15
7. Risks, worst first	16
8. Sources	17

1. Prerequisite

1.1. What you need on the bench

Before starting, get these together:

- An Arduino MKR WAN 1310 with an antenna screwed on. Never transmit without the antenna attached, because the power that should have gone out of the antenna bounces back into the radio chip and can burn it out.
- A second LoRa radio that already works on 869 MHz, so the board has someone to talk to. Any SX1276 or SX1262 board will do. If you use a board that already runs MeshCore, you can reuse it later for the final tests.
- PlatformIO, which is what MeshCore builds with. Version 6.1.19 is what these notes were written against.
- The Arduino `MKRWAN` library. You will not ship it in the final firmware, but you need it early on for the factory reset path.
- Optional but very useful: an RTL-SDR dongle that can tune to 869 MHz. It lets you see whether the board is actually transmitting, without having to trust that the second radio is set up correctly.

1.2. Two brains in one board

The MKR WAN 1310 is unusual. Most Arduino boards have one processor. This one has two, and only one of them is yours.

The first is the **ATSAMD21G18A**. That is the normal Arduino chip, the one your sketch runs on. It has 256 KB of program space and 32 KB of RAM.

The second is hidden inside the silver **Murata CMWX1ZZABZ** can soldered to the board. Inside that can there are two parts: an **STM32L072CZ** processor, and the actual **SX1276** radio chip. The STM32 is wired directly to the radio. The SAMD21 is not.

A useful way to picture it is a taxi. You are sitting in the back seat, which is the SAMD21. The radio is the steering wheel. But the STM32 is the driver, and the driver is the only one who can touch the wheel. You talk to the driver through a small window using short written notes, which in real life are AT commands sent over a serial line. That is exactly what the `MKRWAN` library does.

This arrangement is fine if you want plain LoRaWAN, because the driver already knows the route. It is a problem for MeshCore, which is its own mesh protocol and wants to steer the radio itself, packet by packet. Passing every single instruction through a note in a window is too slow and too limited.

1.3. What we originally planned to do

The original plan was to erase the driver. Write new firmware for the STM32 so that instead of speaking AT commands, it simply relays raw radio instructions from the SAMD21 to the SX1276 and back. In the taxi picture, you would be replacing the driver with a robot who does nothing but copy your hand movements onto the wheel.

That plan would work, but it is a lot of effort. It means writing and maintaining a second piece of firmware, inventing a private protocol between the two chips, and hoping that the delay of passing everything through that relay does not break the radio's timing. It turns out you do not have to.

2. The key discovery: dumb mode

Arduino's own `MKRWAN` library already contains a function that solves this. It is called `dumb()`, and its own comment in the source code says what it is for:

```
CPP
#ifdef SerialLoRa
  // Sends the modem into dumb mode, so the Semtech chip can be
  // controlled directly
  // The only way to exit this mode is through a begin()
  void dumb() {
    SerialLoRa.end();
    pinMode(LORA_IRQ_DUMB, OUTPUT);
    digitalWrite(LORA_IRQ_DUMB, LOW);

    // Hardware reset
    pinMode(LORA_BOOT0, OUTPUT);
    digitalWrite(LORA_BOOT0, LOW);
    pinMode(LORA_RESET, OUTPUT);
    digitalWrite(LORA_RESET, HIGH);
    delay(200);
    digitalWrite(LORA_RESET, LOW);
    delay(200);
    digitalWrite(LORA_RESET, HIGH);
    delay(50);

    // You can now use SPI1 and LORA_IRQ_DUMB as CS to interface
    // with the chip
  }
#endif
```

What this does is hold the STM32 in reset and keep it there. A chip held in reset stops driving its pins, so all the wires it was holding onto go slack. Back to the taxi: `dumb()` does not replace the driver, it asks the driver to get out of the car and hands you the keys. The wheel is now yours.

The important part is the last comment. The wires that run from the radio chip to the STM32 are **also** wired out to the SAMD21. So once the STM32 lets go, the SAMD21 can pick them up and talk to the SX1276 directly over SPI, which is a fast wired bus, not a stream of text commands.

2.1. What this changes about the project

The whole shape of the work changes.

There is no second firmware to write. There is no private protocol to design between the two chips. There is no relay in the middle to slow things down. The STM32 simply sits parked and takes no part.

What is left is this: MeshCore already runs on four families of chip, which are ESP32, nRF52, RP2040 and STM32. It does not yet run on SAMD21. So the real job is to **add**

SAMD21 as a fifth supported chip family, plus one board definition for the MKR WAN 1310. That is ordinary porting work, not radio surgery.

MeshCore also already contains a driver for the SX1276 radio, in `src/helpers/radiolib/CustomSX1276.h`, used by three existing boards. So the radio side is free.

3. What the board gives us, and what it hides

3.1. The pin map

One detail that saves a lot of confused searching: there is **no separate board definition for the 1310**. Both the MKR WAN 1300 and the MKR WAN 1310 share the `mkrwan1300` folder inside Arduino's SAMD core. The handful of pins that only exist on the 1310 are tacked onto the bottom of that same file under a comment that says "MKRWAN1310 compatibility layer". If you go looking for a `mkrwan1310` folder you will not find one.

These are the pins that matter, read out of that file:

What it is	Pin	Name in the header	Notes
Radio chip select	28	<code>LORA_IRQ_DUMB</code>	Doubles as the chip select once you are in dumb mode
SPI clock	37	<code>PIN_SPI1_SCK</code>	On SERCOM4
SPI data out	36	<code>PIN_SPI1_MOSI</code>	On SERCOM4
SPI data in	38	<code>PIN_SPI1_MISO</code>	On SERCOM4
Radio interrupt (DI00)	31	<code>LORA_IRQ</code>	Only exists on the 1310, not the 1300
STM32 reset	30	<code>LORA_RESET</code>	Resets the STM32, not the radio
STM32 BOOT0	33	<code>LORA_BOOT0</code>	Becomes an ordinary pin once the STM32 has booted
Flash chip select	32	<code>FLASH_CS</code>	2 MB memory chip, sharing the same SPI bus
Modem serial line	26 and 29	<code>PIN_SERIAL2_TX</code> and <code>RX</code>	This is the AT command window

3.2. Two things that follow from the wiring

The serial line and the SPI bus cannot both be alive. Inside the SAMD21 there is a block of hardware called SERCOM4 that can be configured either as a serial port or as an SPI bus, but not both at the same time. On this board it is wired to do both jobs. Think of it as a single corridor that either the serial traffic or the SPI traffic can use, and there is no way to widen it.

In practice this costs us nothing, because `dumb()` shuts the serial port down anyway and we never need it again. But it does mean there is no clever fallback where the firmware notices a problem and switches back to AT commands at runtime. Once you go into dumb mode, you are in dumb mode until the board reboots.

The 2 MB memory chip sits on the same bus as the radio. This sounds like a problem and is actually good news. Two devices sharing one SPI bus is completely normal. Each has its own chip select wire, which works like a house number on a street: the bus

shouts the message down the street, and only the house whose number was called answers.

The upside is that MeshCore gets a real place to store things. It needs to remember its own identity key and its list of known contacts across reboots, and 2 MB of dedicated storage is far more comfortable than carving space out of the program memory.

3.3. Three wires we do not get

Holding the STM32 in reset frees up the wires it shared with us. It does not create wires that were never there. Three things stay out of reach.

The radio's own reset line. The SX1276 has a reset pin, but it is wired to the STM32 inside the sealed module and never comes out to the SAMD21. So the radio has no reset button we can press. If it ever locks up, the only cure is to power cycle the whole board. For a repeater sitting on a roof this is worth knowing about, though it is survivable.

The second interrupt line, DI01. The radio has several signal wires it can use to tap the processor on the shoulder. Only one of them, DI00, is wired through to the SAMD21. This matters more than it sounds, and is covered under Phase 4.

The antenna switch controls. This is the big one, and it gets its own section below.

4. Other routes we considered

Two other approaches were looked at and set aside. They are written down because Phase 0 might send us back to one of them.

Run MeshCore on the STM32 instead, and turn the SAMD21 into a plain USB adapter. This avoids the dumb mode gamble entirely, and MeshCore already supports STM32 chips. The reason it is not the first choice is memory. The STM32L072 has only 20 KB of RAM against the SAMD21's 32 KB, and the useful 2 MB memory chip is wired to the SAMD21, not to the STM32, so this route throws away the storage. Keep it as the backup plan.

Ignore the built in radio and wire an external one to the board's normal pin headers. This would give the cleanest radio wiring of all, and would sidestep every question about the antenna switch. It is rejected because it still needs the same SAMD21 porting work, and at that point you are no longer porting MeshCore to the MKR WAN 1310, you are just using it as a generic Arduino with a radio glued on.

5. Phase 0: prove dumb mode actually works

Everything else in this plan depends on one question, and it is worth being blunt about it.

Dumb mode lets go of the SPI wires. Good. But the STM32 was also holding some **other** wires, and we do not automatically know what happens to those. In particular the Murata module contains an **antenna switch**. A radio cannot transmit and receive down the same wire at the same moment, so there is a small electronic switch that points the antenna either at the transmit side or the receive side. It is like a railway junction: the track has to be thrown one way for trains going out and the other way for trains coming in.

Normally the STM32 works that junction. With the STM32 parked, nobody is in the signal box, and the switch is left in whatever position it happens to fall into. There may also be a small precision clock inside the module, called a TCXO, whose power is switched by the STM32.

So the question is not “can the SAMD21 see the radio”, it is “can the SAMD21 see the radio **and** get a working antenna path”. Phase 0 answers that using nothing but Arduino’s stock `LoRa` library. **No MeshCore code gets written until Phase 0 passes.** There is no point porting a protocol stack onto a radio that cannot transmit.

5.1. Step 0.1: establish the way back first

Before changing anything, make sure you can undo it.

Run the `MKRWAN` library’s `FirstConfiguration` example and write down what `modem.version()` and `modem.deviceEUI()` report. Then compile and run `MKRWANFWUpdate_standalone`. That sketch reflashes the STM32 with the original factory firmware, using a small built in bootloader inside the STM32 that listens on the serial line. It is the only route back to a stock board.

Verify it works **now**, while the board is definitely healthy. Finding out that your rescue tool is broken at the same moment you need it is a bad afternoon.

Move on when: the modem answers AT commands, and the restore sketch has been run successfully at least once.

5.2. Step 0.2: knock on the door

This is the cheapest test in the whole project and it settles a surprising amount.

Call `dumb()`, start up SPI1, then read register `0x42` from the SX1276 using pin 28 as the chip select. That register is the chip’s version number and it should read back `0x12`. It is a knock and a password: you ask “who is there” and the right chip answers with the right name.

- If you get `0x12`, then the wiring is right, the chip select is right, and the STM32 really has let go of the bus. Three worries settled in one read.
- If you get `0x00` or `0xFF`, nobody is answering. Either the STM32 is still holding the wires, or the chip select pin is wrong. Stop and fix this before doing anything else.

Read it in a loop for about thirty seconds rather than once. If the value flickers between `0x12` and garbage, the STM32 is waking up now and then and grabbing the bus back, which is a completely different problem from it never letting go.

Move on when: the value reads `0x12`, steadily.

5.3. Step 0.3: transmit

Set the stock `LoRa` library to 869.618 MHz and send a repeating test message.

There is a trap here that wastes a lot of people's time. The SX1276 has two separate output paths for its signal, called RF0 and PA_BOOST. They are two different doors out of the building: one is the main gate and one is a small side door. The Murata module wires its antenna to the PA_BOOST door. If your code sends the signal out of the RF0 door instead, the output is so faint that the radio looks completely dead, even though it is working perfectly.

So if transmission seems to fail, **try the other output path before concluding anything is broken**. In the Sandeep Mistry `LoRa` library this is the second argument to `LoRa.setTxPower()`.

Check the result with the second radio, and with the SDR dongle if you have one. The SDR is worth using here because it proves energy is genuinely leaving the antenna, which the second radio cannot tell you if it happens to be misconfigured.

Move on when: something other than the board under test can see the signal at 869.618 MHz.

5.4. Step 0.4: receive

Now swap roles and have the second radio transmit.

For the first attempt, do not use the interrupt. Instead have the SAMD21 repeatedly read the radio's status register to check whether a packet has arrived. This is the difference between waiting for the doorbell and getting up to check the door every few seconds. Checking manually is wasteful, but it works even if the doorbell wire was never connected, and that is exactly what you want for a first test: it separates a wiring problem on the interrupt line from a real radio problem.

Once packets arrive that way, switch to using the `DI00` interrupt on pin 31 and confirm they still arrive. That tells you the interrupt line is wired the way the header file claims.

Move on when: packets arrive both by checking manually and by interrupt.

5.5. Step 0.5: measure how good the radio path really is

Passing steps 0.3 and 0.4 does **not** prove the antenna switch is in the right position. Two radios sitting on the same desk will hear each other through almost anything. A badly set switch still leaks enough signal to work across a table, then fails completely at fifty metres. You have to measure it.

Put the two boards a fixed distance apart, in a fixed orientation, and do not move them. Send at least fifty packets in each direction and record the signal strength and signal to noise ratio reported for each one. Then get a reference number to compare against, either by measuring the same link using the second radio's own

known good hardware, or by putting the MKR back into normal AT mode and measuring a LoRaWAN transmission at similar settings.

Reading the result:

- **Within about 3 dB of the reference.** The switch is in a usable position. Go to Phase 1.
- **10 to 20 dB below the reference.** Something is wrong with the antenna path. Go to step 0.6.
- **Lopsided, where receiving is fine but transmitting is weak, or the other way round.** This is the signature of a switch stuck pointing one direction. Go to step 0.6.

Keep all these numbers. They are the most interesting measurements in the project and they belong in the final report whether they are flattering or not.

5.6. Step 0.6: the backup plan, a tiny helper firmware

If step 0.5 shows a real loss, the cause is almost certainly the antenna switch sitting in an undefined position because nobody is driving it.

The fix brings back the original idea of writing STM32 firmware, but a far smaller version of it. Instead of a full relay that passes every radio instruction through, you write about fifty lines whose only job is to work the junction. Rather than replacing the driver with a robot who copies everything you do, you are hiring one person whose entire job is to throw one switch.

There is a catch that makes this less simple than it first sounds. **The switch has to move every time the radio changes between sending and receiving.** A helper firmware that just sets the switch once at startup cannot work, because whichever position it picks, the other direction breaks. The helper needs to be told, live, which way to point.

`LORA_BOOT0` on pin 33 is the answer. That pin only has a special meaning during the STM32's startup, when it selects which program to boot. After that it is an ordinary input pin that the SAMD21 can drive high or low whenever it likes. It is also the **only** spare wire available, because pins 26 and 29 belong to the serial port and, as covered earlier, that shares hardware with the SPI bus we are using.

So the helper firmware would:

1. At startup, set up the antenna switch control pins and the TCXO power pin as outputs, and turn the TCXO on. The exact pin numbers have to be read out of the Murata CMWX1ZZABZ datasheet and ST's I-CUBE-LRWAN board configuration. They are deliberately not guessed at here.
2. Set every pin it shares with the SPI bus to a floating input, so the SAMD21 keeps full control of the bus.
3. Set the `BOOT0` pin as an input.
4. Loop forever: read `BOOT0`, point the antenna switch at transmit when it is high and receive when it is low, then sleep until something changes.

On the Arduino side, the firmware raises pin 33 just before transmitting and lowers it just after. That is a small, contained change inside the board definition.

Loading this uses the same mechanism as `MKRWANFWUpdate_standalone`, with your helper firmware swapped in place of the factory image. **Keep the factory image.** It is how you undo this.

There is a neater variant where the helper watches one of the radio's own signal pins to work out when it is transmitting, so the Arduino side needs no changes at all. It is more elegant but it ties the two firmwares together through radio register settings, and it needs a spare signal pin routed to the STM32. Prefer the B00T0 approach unless it turns out to be impossible.

5.7. When Phase 0 is done

All four of these have to be true:

1. Register `0x42` reads `0x12`, steadily.
2. Packets travel in both directions at 869.618 MHz.
3. The link measures within roughly 3 dB of the reference, with or without the helper firmware.
4. The factory firmware restore has been proven to work.

6. Phases 1 to 5

6.1. Phase 1: teach MeshCore's build system about SAMD21

Add a `samd21_base` section to `platformio.ini` next to the existing ones for ESP32, nRF52, RP2040 and STM32. It needs the `atmelsam` platform, the `mkrwan1310` board, and a `SAMD21_PLATFORM` flag so the code can tell which chip it is on. Build it on top of the existing `arduino_base` section so the shared settings carry over.

The goal here is only to get it to **compile**, not to work. Expect to spend the time on places where the code already branches on chip family, such as `CustomSX1276.h`, which currently has special cases for nRF52 and RP2040 and will need one more.

The real output of this phase is a **number**: how much program space and how much RAM MeshCore actually uses on this chip.

That number is currently unknown. An attempt was made to get a rough idea by building MeshCore for the Wio E5, which uses a similar small processor, but that build failed for a reason that has nothing to do with this project: RadioLib needs a header called `SubGhz.h` that the STM32 platform package does not ship. So there is no estimate, only a measurement waiting to be taken.

Move on when: it compiles and links, and the memory figures are written down. If RAM use is above roughly 24 KB, stop and seriously consider the STM32 backup plan before building anything else on top.

6.2. Phase 2: board support and storage

Write a `SAMD21Board` class implementing the interface MeshCore expects from a board, covering things like battery voltage, reset and timekeeping. The existing `STM32Board.h` and `NRF52Board.h` are the models to copy from.

Set up a filesystem on the 2 MB memory chip using chip select pin 32, so MeshCore has somewhere to keep its identity and contacts. Remember it shares the bus with the radio, and RadioLib will be using that bus from inside interrupt handlers. Writing to storage from an interrupt, or interrupting a storage write with a radio transaction, is the kind of bug that shows up once an hour and takes a week to find. Get the bus locking right the first time.

One loose end: battery measurement. On the 1310, the pin that would normally read the battery voltage has been reused as the flash chip select. The header file says so outright. So battery reporting either needs another route or has to be left unimplemented.

6.3. Phase 3: the board definition

Create `variants/mkrwan1310/` following the existing `variants/heltec_v2/`, which is the closest board that also uses an SX1276.

The radio settings come straight from the pin table:

INI

```
-D RADIO_CLASS=CustomSX1276
-D WRAPPER_CLASS=CustomSX1276Wrapper
-D P_LORA_NSS=28
-D P_LORA_SCLK=37
-D P_LORA_MOSI=36
-D P_LORA_MISO=38
-D P_LORA_DIO_0=31
-D P_LORA_DIO_1=RADIOLIB_NC
-D P_LORA_RESET=RADIOLIB_NC
-D LORA_FREQ=869.618
```

The two `RADIOLIB_NC` entries are the wires we established do not exist. `NC` means not connected, and it tells RadioLib not to wait for signals that will never come.

The startup code performs the dumb mode sequence before setting the radio up. **Write those lines out directly rather than depending on the `MKRWAN` library.** It is about a dozen calls to `pinMode` and `digitalWrite`, and pulling in the entire modem library to reach one small function would drag the whole AT command parser into a build that is already short of space. Put a comment above it pointing at where the sequence came from.

Transmit power has to use the `PA_BOOST` path found in step 0.3, and has to stay inside the legal limits for the 869 MHz band, which include a duty cycle restriction on how much of each hour you are allowed to spend transmitting.

6.4. Phase 4: fix the channel check

This is the consequence of DIO1 not being wired, and it is worth explaining because the failure is silent.

Before transmitting, a well behaved radio listens to check nobody else is already talking. MeshCore does this in a function called `tryScanChannel()`. That function watches two of the radio's signal wires: DIO0 tells it the check has finished, and DIO1 tells it whether anything was heard.

On this board DIO1 is not connected, so the code reads it as permanently low, which it interprets as "nothing heard". The result is that the board will believe the channel is clear every single time, and will transmit straight over the top of other traffic. Nothing crashes. It just quietly becomes a rude neighbour, and only shows up as poor performance once there are several nodes in range.

The fix is to write a version of `tryScanChannel()` that reads the answer out of the radio's status register over SPI instead of watching the missing wire. It is the same doorbell problem from step 0.4: the wire is not there, so go and check the door yourself. Slightly slower, and correct.

For the first working build, target the `simple_repeater` firmware. The `companion_radio` build, which is the one that talks to a phone, keeps large contact tables in memory and is not a realistic fit for 32 KB. Get the repeater working first.

6.5. Phase 5: real world testing and write up

Test against a known good MeshCore node: do adverts propagate, does it forward messages across more than one hop, does it stay up for days rather than hours, and how does its signal strength compare with the reference numbers taken back in step 0.5.

Then write it all up. The measurements from step 0.5 and the memory figures from Phase 1 are the substance of the report. Include the ones that did not go well too, since a port that documents where it fell short is more useful than one that only lists what worked.

7. Risks, worst first

The antenna path may simply not work in dumb mode. The switch and possibly the TCXO are left unattended when the STM32 is parked. This could show up as total silence, or as a quiet 15 dB loss that only appears once the boards are far apart. Step 0.5 is designed to catch it and step 0.6 is the fix. This single risk is the reason Phase 0 exists and the reason no MeshCore code is written before it passes.

Listen before transmit will be broken by default. Confirmed by reading the header file, not guessed. The fix is known and written up in Phase 4, so this is low risk, but it must not be forgotten because it fails quietly.

32 KB of RAM may not be enough. MeshCore's smallest current target has twice that. This is unmeasured, and Phase 1 exists to measure it. If it does not fit, the options in order are: shrink the contact and neighbour limits, build only the repeater firmware, or fall back to running on the STM32 instead.

The radio cannot be reset in software. No wire, no fix. A locked up radio needs the power removed. Acceptable for a repeater, but it should be stated plainly in the report.

The radio and the storage chip share one bus. Normal engineering, but RadioLib works from interrupt context, so filesystem access has to be protected. Watch for it during Phase 2.

The Murata module can be left unresponsive. Any time you write firmware into the STM32 there is a chance of ending up with a module that does not answer. This is why step 0.1 proves the restore path works before step 0.6 ever writes anything.

8. Sources

- [MKRWAN library, the `dumb\(\)` function](#)
- [MKRWANFWUpdate_standalone, the factory restore sketch](#)
- [ArduinoCore-samd, the mkrwan1300 pin definitions](#)
- [arduino/mkrwan1300-fw, the factory Murata firmware](#)
- [MKRWAN issue 76, on the serial and SPI clash](#)
- [Arduino MKR WAN 1310 datasheet](#)
- [Upgrading the Murata LoRa module firmware](#)